

Tarea de laboratorio 2 – Pilas (S02)

Objetivos

- Practicar en Java la utilización de las pilas como estructura de datos, manipulando varias pilas de forma coordinada.

Descripción de la práctica:

En muchos juegos de cartas coleccionables (*Magic: The Gathering*, *Slay the Spire*, etc.) existe la mecánica de la **Pila de Efectos** (*Effects Stack*): cuando un jugador lanza un hechizo o acción, este no se resuelve inmediatamente, sino que se apila en una pila global. Otros jugadores pueden responder lanzando sus propias cartas encima. Cuando se decide resolver la jugada, los efectos se ejecutan **desde el último que se jugó hasta el primero**.

El objetivo es desarrollar un programa en Java que implemente un motor de simulación de turnos de una partida multijugador de un juego de cartas, leyendo las acciones que se deberán simular de un archivo de entrada `acciones.csv`.

Las cartas del juego se representarán mediante la siguiente enumeración en Java:

```
public enum Carta {GOLPE_CRITICO, RAYO_CONGELANTE, BARRERA_DE_HIELO, POCION_DE_VIDA,
CONTRAHECHIZO, ESCUDO_MAGICO, BOLA_DE_FUEGO}
```

El motor de simulación de turnos debe gestionar tres pilas:

1. **Mazo de robo:** Contiene las cartas disponibles para robar. Al iniciar la simulación se deberá añadir una carta de cada uno de los tipos indicados anteriormente a este mazo, siguiendo el orden indicado en la enumeración.
2. **Mazo de descartes:** Contiene las cartas que han sido usadas o descartadas.
3. **Pila de Efectos:** Pila global donde se acumulan las cartas de hechizo pendientes de resolución. Cada elemento en esta pila deberá representar no solo la Carta jugada, sino también el jugador que la lanzó.

El formato de las acciones especificadas en `acciones.csv` es el siguiente: Se tienen tres cadenas separadas por comas, la primera indica el jugador, la segunda la acción, y la tercera la carta correspondiente. Cuando la acción esté relacionada con una carta (JUGAR o DESCARTAR), la tercera columna indicará el valor de la carta. En este caso el valor en el CSV corresponderá con uno de los valores en la enumeración `Carta`. Cuando la acción no requiera una carta (caso de ROBAR o RESOLVER), no habrá una tercera cadena. A continuación, se muestra un ejemplo sencillo de posible contenido del archivo `acciones.csv`.

```
Jugador 1,ROBAR,
Jugador 2,ROBAR,
Jugador 1,JUGAR,BOLA_DE_FUEGO
Jugador 2,JUGAR,ESCUDO_MAGICO
Sistema,RESOLVER,
```

Las reglas de ejecución de acciones se indican a continuación:

1. **ROBAR:**
 - Se extrae una carta del mazo de robo.

- **Regla de Reciclaje:** Si el mazo de robo está vacío en el momento de robar, se deben traspasar automáticamente todas las cartas del mazo de descartes al mazo de robo, una a una. Si ambos mazos están vacíos, se notificará un mensaje de error.
- 2. **JUGAR:**
 - Añade un efecto (representado por la carga jugada y el jugador que la juega) a la pila de efectos.
- 3. **DESCARTAR:**
 - Añade la carta descartada directamente al mazo de descartes.
- 4. **RESOLVER:**
 - Procesa **todos** los efectos de la pila de efectos, extrayéndolos uno a uno.
 - Al resolverse cada efecto, se debe mostrar por pantalla el nombre de la carta y el jugador que la jugó, y trasladar dicha carta al mazo de descartes.

El motor de simulación procesará cada acción en el CSV, y para cada una de ellas mostrará en pantalla la siguiente información: descripción de la acción procesada (jugador, acción, carta), detalle del evento para acciones de tipo RESOLVER o ROBAR cuando se recicla el mazo de descartes en el de robo), y estado actualizado de los mazos y la pila de efectos tras realizar la acción.

Requisitos técnicos

- Los programas no podrán contener moldes en relación con las pilas ni ninguna referencia a un `Object`.
- Se deberá utilizar la implementación de Pilas (`Stack`) en la librería estándar de Java.
- Se deberá realizar un tratamiento adecuado de errores y excepciones: control de archivos no encontrados, acciones y cartas no reconocidas, etc.
- Cada clase del programa y método relevante deberá contener documentación interna. En Campus Virtual se puede encontrar información para generar esta documentación (sección *Laboratorio*). Además, se deberá seguir una modularización y un diseño orientado a objetos adecuado.
- Con el fin de utilizar sistemas antiplagio y de detección semántica de similitudes, los ficheros de código fuente no podrán contener datos sobre los estudiantes de carácter personal como el nombre, el correo electrónico... En su lugar, se identificará a los autores y el grupo de prácticas en los comentarios del código con las iniciales de los estudiantes y el código de equipo asignado.
- No se podrán utilizar caracteres especiales en los nombres de los ficheros.

Normas de realización

- La práctica deberá realizarla cada uno de los equipos de trabajo formados previamente en clase de laboratorio. No podrá realizarse de forma individual.
- Por motivos pedagógicos, no estará permitido utilizar asistentes inteligentes de generación de código (programas), como Chat-GPT, GitHub Copilot o Claude, y se penalizarán aquellas prácticas que se alejen de la filosofía de la programación estructurada y la programación orientada a objetos.